

Chapter 1 — Entity Framework Core Fundamentals and Installation

1. Introduction to Entity Framework Core

Entity Framework Core (EF Core) is a **lightweight, open-source, cross-platform Object-Relational Mapper (ORM)** for .NET. It enables developers to interact with relational databases using **C# objects** instead of writing SQL queries directly.

EF Core is the **successor to Entity Framework 6** and is designed to work with:

- .NET Core
- .NET 5/6/7+
- .NET Framework 4.6+

Core Purpose

EF Core simplifies database operations by automatically mapping:

Database Concept	EF Core Equivalent
Table	C# Class (Entity)
Row	Object
Column	Property
Primary/Foreign Keys	Relationships

Through this mapping, developers can perform CRUD and query operations in C# while EF Core internally generates the SQL.

2. Why Use an ORM like EF Core?

ORMs allow database interaction through objects instead of SQL scripts.

Key Benefits

1. **Reduces Manual SQL Coding**
CRUD logic is handled via EF Core APIs.
2. **Compile-Time Type Safety**
Errors are detected during development rather than runtime.
3. **Improved Productivity and Maintainability**
Code becomes cleaner and easier to manage.
4. **Database Independence**
The same codebase can support multiple relational databases.
5. **Advanced Features Provided Automatically**
 - Lazy Loading
 - Change Tracking
 - Caching
 - Connection Pooling

6. Reduced Impact of Schema Changes

Refactoring entity classes updates corresponding mappings.

3. Key Features of EF Core

EF Core provides the following important capabilities:

- Cross-platform support (Windows, Linux, macOS)
 - Lightweight and extensible architecture
 - Supports Code-First and Database-First approaches
 - Strongly-typed LINQ querying
 - Performance-optimized execution
 - Rich relationship handling:
 - One-to-One
 - One-to-Many
 - Many-to-Many
 - Schema evolution via **Migrations**
 - Change Tracking & Entity State Management
 - Lazy & Explicit Loading support
- 

4. EF Core Development Approaches

A. Code-First Approach (Most Common)

In this approach, the **database is generated from C# classes**.

You create:

- Entity Classes (POCOs)
- A DbContext class

EF Core uses conventions to generate tables automatically.

This is recommended for:

- ✓ New applications
 - ✓ Agile development environments
 - ✓ Domain-driven designs
-

B. Database-First Approach

In this approach, an **existing database is reverse-engineered into C# entity classes and DbContext**.

EF Core scaffolds model classes from the database.

Used when:

- ✓ A legacy database already exists
- ✓ Schema must not change frequently

5. Database Providers in EF Core

EF Core communicates with databases using **providers**, which are database-specific libraries.

Examples of Providers

Database	Provider Package
SQL Server	Microsoft.EntityFrameworkCore.SqlServer
SQLite	Microsoft.EntityFrameworkCore.Sqlite
PostgreSQL	Npgsql.EntityFrameworkCore.PostgreSQL
MySQL	Pomelo.EntityFrameworkCore.MySql

A **provider acts as a bridge** between EF Core and the database engine.

6. Installing EF Core (SQL Server Example)

EF Core is **not included by default**. It must be installed using NuGet.

Required Packages

Install the following packages:

Microsoft.EntityFrameworkCore

Microsoft.EntityFrameworkCore.SqlServer

Microsoft.EntityFrameworkCore.Tools

Microsoft.EntityFrameworkCore.Design

Purpose of Each Package

Package	Purpose
Microsoft.EntityFrameworkCore	Core runtime for EF Core
Microsoft.EntityFrameworkCore.SqlServer	Enables SQL Server support
Microsoft.EntityFrameworkCore.Tools	Enables migrations and CLI tools
Microsoft.EntityFrameworkCore.Design	Supports design-time features such as scaffolding

The **Design package is optional** if you do not use migrations.

7. DbContext — Core Component of EF Core

DbContext represents a session with the database.

It manages:

- Database connections
- Query execution
- Saving changes
- Change tracking
- Model creation
- Transaction handling
- Entity caching

It is the **primary communication bridge between your application and the database.**

Example — DbContext Configuration (Without Dependency Injection)

protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)

```
{  
    optionsBuilder.UseSqlServer(  
        @"Server=smvitanet;Database=SchoolDB;Trusted_Connection=True;");  
}
```



8. DbSet — Table Representation

DbSet<TEntity> represents a **table in the database** and provides APIs for CRUD operations.

Example:

```
public DbSet<Student> Students { get; set; }
```

DbSet acts as:

- ✓ Table Mapping
- ✓ Query Interface
- ✓ CRUD Gateway

9. Change Tracking and Entity States

EF Core automatically monitors entity changes to determine which SQL commands should be executed.

Entity States

State	Description
Added	A new row will be inserted
Modified	Existing row will be updated
Deleted	Row will be removed
Unchanged	No update required

Tracking begins when:

- Entities are loaded
- Entities are added
- Entities are attached manually

This forms the basis for **SaveChanges()** execution.

10. Strong Typing and LINQ Querying

EF Core supports **strongly-typed LINQ queries**, allowing:

- ✓ Compile-time validation
- ✓ Auto-completion support
- ✓ Safer and cleaner code

EF Core converts LINQ expressions into SQL at runtime.

11. Example — Model Classes (POCOs)

Grade Entity

```
public class Grade
{
    public int Id { get; set; }
    public string GradeName { get; set; }
    public string Section { get; set; }
    public IList<Student> Students { get; set; }
}
```

Student Entity

```
public class Student
{
    public int StudentId { get; set; }
    public string FirstName { get; set; }
    public string LastName { get; set; }
    public DateTime DateOfBirth { get; set; }
    public float Height { get; set; }
    public float Weight { get; set; }
    public int GradeId { get; set; }
}
```

```
public Grade Grade { get; set; }  
}
```

This defines a **One-to-Many relationship**:

- One Grade
 - Many Students
-

12. EF Core Architecture Overview

Application → DbContext → EF Core Provider → Database

This layered flow ensures abstraction and maintainability.



Chapter 2 — DbContext, DbSet, Entity States & CRUD Operations

1. Role of DbContext in EF Core

DbContext is the **central class** in EF Core. It represents a **session with the database**, meaning that it:

- Manages the database connection
- Tracks entity changes
- Generates SQL commands
- Executes queries
- Manages transactions
- Maps models to tables

It acts as the **bridge between application code and the database engine**.

Typical DbContext Structure

```
public class SchoolContext : DbContext
{
    protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
    {
        optionsBuilder.UseSqlServer(
            @"Server=smvitanet;Database=SchoolDB;Trusted_Connection=True;");
    }

    public DbSet<Student> Students { get; set; }
    public DbSet<Grade> Grades { get; set; }
}
```



2. DbSet — Table Mapping and Query Interface

DbSet<TEntity> represents a **table** and provides:

- ✓ Querying capability
- ✓ CRUD operations
- ✓ Change tracking support

For example:

```
public DbSet<Student> Students { get; set; }
```

This means:

Table in DB → Students

Class in code → Student

DbSet behaves like an **in-memory collection** while also being a **gateway to the database**.

3. Entity States & Change Tracking

EF Core automatically tracks entity state to determine the correct SQL operation when `SaveChanges()` is executed.

Entity States

State	Meaning	SQL Operation
Added	New entity created	INSERT
Modified	Existing entity changed	UPDATE
Deleted	Entity marked for removal	DELETE
Unchanged	No change detected	None

Tracking begins when:

- An entity is loaded from the DB
- An entity is added to DbSet
- An entity is attached manually

Why This Matters

EF Core automatically determines the **correct SQL operation**, so developers do not need to write SQL manually.

4. CRUD Operations Using EF Core

CRUD = Create, Read, Update, Delete.

All operations are performed through DbContext & DbSet.

A. INSERT — Adding Data

Student s = new Student

```
{  
    FirstName = "Geeta",  
    LastName = "Zaveri",  
    GradeId = 1  
};
```

```
context.Students.Add(s);
```

```
context.SaveChanges();
```

Process Explanation

1. Entity marked as **Added**
 2. EF Core generates INSERT SQL
 3. Record is stored in database
-

B. READ — Retrieving Data

Get All Records

```
var students = context.Students.ToList();
```

Get by Primary Key

```
var student = context.Students.Find(1);
```

LINQ Query

```
var data = context.Students  
    .Where(s => s.GradeId == 1)  
    .ToList();
```

EF Core converts LINQ → SQL automatically.

C. UPDATE — Modifying Data

```
var s = context.Students.Find(1);  
s.LastName = "Deshmukh";  
context.SaveChanges();
```

Explanation

1. Entity becomes **Modified**
2. EF Core generates UPDATE SQL
3. Updates DB record



D. DELETE — Removing Data

Delete Fetch + Remove

```
var s = context.Students.Find(1);  
context.Students.Remove(s);  
context.SaveChanges();
```

Stub Delete (No Fetch Needed)

```
var s = new Student { StudentId = 1 };  
context.Students.Attach(s);  
context.Students.Remove(s);  
context.SaveChanges();
```

This is efficient when you already know the ID.

5. SaveChanges() — The Commit Operation

SaveChanges() sends all tracked entity changes to the database in a **single transaction**.

State	SQL Executed
Added	INSERT
Modified	UPDATE
Deleted	DELETE

This means:

- ✓ Multiple changes → One Commit
 - ✓ Ensures data integrity
-

6. Connected vs Disconnected Scenarios

Connected

Same DbContext instance is used to:

- ✓ Load entity
- ✓ Modify
- ✓ Save

EF Core automatically detects changes.

Disconnected

Occurs in:

- Web applications
- APIs
- Distributed systems

Entity is **retrieved in one context and updated in another**.

In such cases, we may need to call:

```
context.Attach(entity);
```

```
context.Entry(entity).State = EntityState.Modified;
```

This informs EF Core that the entity should be updated.

7. Cascade Delete & Relationships (High-Level View)

When a parent entity is deleted:

- ✓ Dependent entities may also delete automatically
- ✓ This is called **Cascade Delete**

Example:

Deleting Grade may delete Students if configured.

If cascade is **disabled**, database will throw an error unless dependents are handled manually.

8. Strong Typing & LINQ — Key Advantages

- ✓ Queries validated during compilation
- ✓ Lower runtime errors
- ✓ IntelliSense support
- ✓ Cleaner syntax

Example:

```
var result = context.Students
    .Where(s => s.Height >= 5)
    .ToList();
```

EF Core translates this to SQL automatically.

9. Example — Simple Data Access Layer

```
class SchoolGradeDAL
{
    SchoolContext db = new SchoolContext();
    public List<Student> GetAll()
    {
        return db.Students.ToList();
    }
    public void AddStudent(Student s)
    {
        db.Add(s);
        db.SaveChanges();
    }
}
```



10. Understanding First(), Single(), and Default Variants

Method	Expected Records	Behavior
Single()	Exactly 1	Throws if 0 or >1
SingleOrDefault()	0 or 1	Returns null if none
First()	At least 1	Throws if none
FirstOrDefault()	0 or more	Returns null if none

Performance Note:

First() is slightly faster because it fetches only **TOP 1**.

Chapter 3 — Relationships, Conventions & Configuration in Entity Framework Core (Data Annotations and Fluent API)

1. Purpose of Configuration in EF Core

Entity Framework Core applies **default conventions** to map classes to database schema.

Configuration is used when you need to:

- Override conventions
- Control table and column names
- Define relationships explicitly
- Apply validation and constraints
- Configure database behavior
- Improve clarity and maintainability

Configuration can be done using:

1. **Data Annotations (Attributes on classes/properties)**
2. **Fluent API (Inside OnModelCreating in DbContext)**

2. EF Core Conventions — Default Mapping Rules

A. Primary Key Convention

EF Core treats a property as **Primary Key** automatically if named:

- Id
- <EntityName>Id

Examples:

```
public class Book
{
    public int Id { get; set; } // Primary Key
}
```

or

```
public class Book
{
    public int BookId { get; set; } // Primary Key
}
```

If both exist, **Id is preferred**.

B. Foreign Key Convention

A property is treated as a **Foreign Key** if named as:

- <PrincipalClassName>Id
- <PrimaryKeyName>

Example:

```
public class Author
{
    public int AuthorId { get; set; }
    public ICollection<Book> Books { get; set; }
}

public class Book
{
    public int BookId { get; set; }
    public int AuthorId { get; set; } // FK
    public Author Author { get; set; } // Navigation
}
```

Foreign key rules:

- ✓ Nullable FK → Optional relationship
- ✓ Non-nullable FK → Required relationship

C. Table and Column Mapping Convention

- Class name → Table name
- Property name → Column name
- Default schema → dbo

If a DbSet exists:

```
public DbSet<Book> Books { get; set; }
```

Then table name defaults to **Books**.

D. Shadow Foreign Keys

If FK is not declared explicitly, EF Core creates one internally (shadow property), e.g.,

AuthorId

3. Relationship Conventions

A. One-to-Many (Most Common)

Occurs when principal contains a **collection navigation**.

Example:

```
public class Author
{
    public int AuthorId { get; set; }

    public ICollection<Book> Books { get; set; }
}
```

EF Core will:

- ✓ Create AuthorId in Books table
- ✓ Establish FK relationship

B. One-to-One Relationship

Identified when:

- Both entities have navigation properties
- Dependent contains FK

Example:

```
public class Author
{
    public int AuthorId { get; set; }

    public AuthorBiography Biography { get; set; }
}
```



```
public class AuthorBiography
{
    public int AuthorBiographyId { get; set; }

    public int AuthorId { get; set; } // FK

    public Author Author { get; set; }
}
```

EF Core will:

- ✓ Create **unique index** on AuthorId
- ✓ Enforce 1:1 pairing

If FK is missing or mis-named, Fluent API must configure it.

C. Delete Behaviors (Referential Actions)

When principal entity is deleted, EF Core may:

Behavior	Meaning
Restrict / NoAction (Default)	Prevent delete if dependents exist
Cascade	Delete dependents automatically
SetNull	Set FK to NULL
SetDefault	Set FK to default value

Cascade delete **automatically applies when FK is required**.

4. Configuring Models with Data Annotations

Data Annotations are applied **directly on classes and properties**.

They come from:

System.ComponentModel.DataAnnotations

System.ComponentModel.DataAnnotations.Schema

A. Table Mapping

[Table("tbl_Book")]

public class Book { }

Schema + table:

[Table("Books", Schema = "library")]

public class Book { }

B. Column Configuration

[Column("Description", TypeName = "nvarchar(100)"]

public string Title { get; set; }

C. Primary Key

[Key]

public int OrderNumber { get; set; }

Composite keys **cannot be defined via attributes** (must use Fluent API).

D. Foreign Key Mapping

```
public class Book
{
    public int BookId { get; set; }

    [ForeignKey("AuthorFK")]
    public Author Author { get; set; }

    public int AuthorFK { get; set; }
}
```

E. Required Field

```
[Required]
public string Title { get; set; }
Creates NOT NULL constraint.
```

F. Max Length and String Length

```
[MaxLength(150)]
public string Title { get; set; }
```

G. Indexing (.NET 5+)

```
[Index(nameof(Isbn), IsUnique = true)]
public class Book
{
    public string Isbn { get; set; }
}
```

H. Ignore Property

```
[NotMapped]
public string FullName { get; set; }
```

5. Fluent API Configuration

Fluent API is defined inside:

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
```

Use Fluent API when:

- ✓ Relationships are complex
- ✓ Composite keys are needed
- ✓ Naming conventions differ
- ✓ Configuration must be centralized

A. Define Composite Key

```
modelBuilder.Entity<Order>()  
.HasKey(o => new { o.OrderId, o.LineId });
```

B. Configure One-to-Many Relationship

```
modelBuilder.Entity<Book>()  
.HasOne(b => b.Author)  
.WithMany(a => a.Books)  
.HasForeignKey(b => b.AuthorId);
```

C. Configure One-to-One Relationship

```
modelBuilder.Entity<Author>()  
.HasOne(a => a.Biography)  
.WithOne(b => b.Author)  
.HasForeignKey<AuthorBiography>(b => b.AuthorId);
```

D. Configure Table

```
modelBuilder.Entity<Book>()  
.ToTable("tbl_Book");
```

E. Configure Property

```
modelBuilder.Entity<Book>()  
.Property(b => b.Title)  
.HasMaxLength(150)  
.IsRequired();
```

6. Data Annotations vs Fluent API — Comparison

Feature	Data Annotations	Fluent API
Simple property rules	✓ Yes	✓ Yes
Complex relationships	✗ Limited	✓ Best
Composite keys	✗ No	✓ Yes
Centralized configuration	✗ No	✓ Yes
Readability	✓ High	Moderate

Best Practice:

Use **Data Annotations** for simple rules and **Fluent API** for relationships and keys.



Chapter 4 — Migrations, Repository Pattern & Dependency Injection in Entity Framework Core

1. Purpose of Migrations in EF Core

In **Code-First development**, the database schema is generated from C# model classes.

As your application evolves, models change — and the database must remain synchronized.

Migrations provide a controlled mechanism to:

- Create the initial database
- Apply schema updates incrementally
- Track version history
- Avoid manual SQL modification

This ensures **database schema remains consistent with your model classes**.

2. Required EF Core Packages for SQL Server

EF Core is installed via NuGet packages:

Microsoft.EntityFrameworkCore

Microsoft.EntityFrameworkCore.SqlServer

Microsoft.EntityFrameworkCore.Tools

Microsoft.EntityFrameworkCore.Design

Role of Packages

Package	Purpose
Microsoft.EntityFrameworkCore	Core EF runtime
Microsoft.EntityFrameworkCore.SqlServer	SQL Server provider
Microsoft.EntityFrameworkCore.Tools	Enables migrations
Microsoft.EntityFrameworkCore.Design	Required for scaffolding & design-time work

The **Design package** is not mandatory if migrations are not used.

3. Creating Database using Migrations

Step 1 — Add Migration

Add-Migration CreateSchoolDB

Creates a migration script based on your models.

Step 2 — Apply Migration

Update-Database -Verbose

This command:

- ✓ Creates database (if not present)
 - ✓ Applies schema changes
 - ✓ Displays executed SQL (-Verbose)
-

4. DbSet and DbContext — Repository-Like Behavior

DbSet<TEntity> already behaves like a **repository**, because it:

- ✓ Represents a table
- ✓ Provides CRUD APIs
- ✓ Supports LINQ
- ✓ Handles tracking

Example DbContext:

```
public class SampleContext : DbContext
{
    protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
    {
        optionsBuilder.UseSqlServer(
            @"Data Source=(localdb)\ProjectModels;
            Initial Catalog=DemoData;
            Integrated Security=True;");
    }

    public DbSet<Book> Books { get; set; }
    public DbSet<Author> Authors { get; set; }
}
```



5. Repository Pattern in EF Core

Although DbSet already functions like a repository, many architectures introduce an **explicit repository layer** to:

- Abstract EF Core dependency
 - Improve testability
 - Enforce separation of concerns
 - Centralize data access logic
-

Repository Interface Example

```
internal interface ISqlRepository
{
    List<Book> GetAll();

    Book GetById(int id);

    Book DeleteById(int id);

    Book AddBook(Book book);

    Book Modify(Book book);
}
```

Repository Implementation

```
internal class SqlService : ISqlRepository
{
    SampleContext Context;

    public SqlService(SampleContext context)
    {
        Context = context;
    }

    public Book AddBook(Book book)
    {
        Context.Books.Add(book);
        Context.SaveChanges();
        return book;
    }

    public Book DeleteById(int id)
    {
        Book b = Context.Find<Book>(id);
        Context.Remove(b);
        Context.SaveChanges();
        return b;
    }

    public List<Book> GetAll()
    => Context.Books.ToList();

    public Book GetById(int id)
    => Context.Find<Book>(id);

    public Book Modify(Book book)
    {

```



```
        Book b = Context.Find<Book>(book.BookId);
        Context.Update(b);
        Context.SaveChanges();
        return book;
    }
}
```

6. Controller-Like Service Layer

```
internal class BookControllerDemo
{
    ISqlRepository Service;

    public BookControllerDemo(ISqlRepository service)
    {
        Service = service;
    }

    public Book Add(Book book) => Service.AddBook(book);
    public List<Book> GetBooks() => Service.GetAll();
    public Book Delete(int id) => Service.DeleteById(id);
    public Book Put(Book book) => Service.Modify(book);
}
```

This layer **decouples UI / API from data access.**

7. Dependency Injection in EF Core

Key Terms

Term	Meaning
IServiceCollection	Registers dependencies
IServiceProvider	Resolves dependencies

Register Dependencies

```
var services = new ServiceCollection();

services.AddTransient<ISqlRepository, SqlService>();
services.AddTransient<BookControllerDemo>();
services.AddTransient<SampleContext>();

var serviceProvider = services.BuildServiceProvider();
```

Resolve Dependency

```
var controller = serviceProvider.GetService<BookControllerDemo>();
```

Execution Example

```
Book obj1 = new Book { Title = "New Thinking in Java", AuthorId = 2 };
```

```
controller.Add(obj1);
```

```
controller.Delete(3);
```

```
var booklist = controller.GetBooks();
```

```
foreach (Book b in booklist)
```

```
    Console.WriteLine(b.Title);
```

8. Dependency Injection — Key Concepts

- ✓ Controller depends on interface
- ✓ Repository injects DbContext
- ✓ DbContext connects to database
- ✓ Objects created automatically

This results in:

- ✓ Loose coupling
 - ✓ Improved maintainability
 - ✓ Easy testing
 - ✓ Clean architecture
-

9. DI Lifetime Keywords (Important)

Lifetime	Meaning
AddTransient	New instance each request
AddScoped	One instance per HTTP request
AddSingleton	One instance for entire app

DbContext is usually **Scoped** in web apps.

Chapter 5 — MVC CRUD with Entity Framework Core: End-to-End Execution Flow

1. Overview of MVC + EF Core CRUD Architecture

The typical dependency flow is:

Controller → Service / Repository → DbContext → Database

Where:

- ✓ Controller handles HTTP requests
 - ✓ Service/Repository performs business & data logic
 - ✓ DbContext communicates with database
 - ✓ EF Core executes SQL automatically
-

2. Create POCO (Model) Classes

Models represent database tables.

Example:

```
public class Employee
{
    public int EmployeeId { get; set; }
    public string Name { get; set; }
    public decimal Salary { get; set; }
    public int DepartmentId { get; set; }

    public Department Department { get; set; }
}
```

Validation attributes may also be applied here (e.g., [Required], [MaxLength]).

3. Create DbContext Class

Your DbContext must inherit from DbContext.

```
public class AppDbContext : DbContext
{
    public AppDbContext(DbContextOptions<AppDbContext> options)
        : base(options)
    {
    }

    public DbSet<Employee> Employees { get; set; }
    public DbSet<Department> Departments { get; set; }
}
```

DbSet represents tables.

4. Add Connection String in appsettings.json

Example:

```
"ConnectionStrings": {  
  "EmployeeDBConnection": "Data Source=(localdb)\\Projects;Initial Catalog=DemoDB;Integrated Security=True"  
}
```

This enables secure configuration without hard-coding credentials.

5. Register Services in Program.cs

```
builder.Services.AddDbContextPool<AppDbContext>(  
    options => options.UseSqlServer(  
        builder.Configuration.GetConnectionString("EmployeeDBConnection")  
    ));  
  
builder.Services.AddScoped<IEmployeeService, SqlEmployeeService>();  
  
builder.Services.AddControllersWithViews()  
    .AddJsonOptions(o =>  
        o.JsonSerializerOptions.ReferenceHandler = ReferenceHandler.IgnoreCycles);
```

Key Points:

- ✓ DbContext is injected into the pipeline
 - ✓ Repository/Service is registered
 - ✓ JSON cycle handling avoids circular reference issues in API responses
- 

6. Add EF Core Migrations

Command 1 — Generate Migration

Add-Migration CreateDatabase

Command 2 — Apply Migration

Update-Database -Verbose

Result:

- ✓ Database + Tables created from models
 - ✓ Schema version stored
 - ✓ SQL executed transparently
-

7. Implement Repository / Service Layer

Interfaces abstract data access:

```
public interface IEmployeeService
```

```
{  
    List<Employee> GetAll();  
    Employee GetById(int id);  
    Employee Add(Employee e);  
    Employee Update(Employee e);  
    Employee Delete(int id);  
}
```

Implementation uses EF Core:

```
public class SqlEmployeeService : IEmployeeService
```

```
{  
    private readonly AppDbContext _context;  
  
    public SqlEmployeeService(AppDbContext context)  
    {  
        _context = context;  
    }
```

```
    public List<Employee> GetAll()  
    => _context.Employees.ToList();
```

```
    public Employee GetById(int id)  
    => _context.Employees.Find(id);
```

```
    public Employee Add(Employee e)  
    {  
        _context.Employees.Add(e);  
        _context.SaveChanges();  
        return e;  
    }
```

```
    public Employee Update(Employee e)  
    {  
        _context.Employees.Update(e);  
        _context.SaveChanges();  
        return e;  
    }
```

```
    public Employee Delete(int id)  
    {  
        var e = _context.Employees.Find(id);  
        _context.Employees.Remove(e);  
        _context.SaveChanges();  
        return e;  
    }
```

```
}
```

This centralizes CRUD logic.



8. Create MVC Controller

Controller uses constructor injection:

```
public class EmployeeController : Controller
{
    private readonly IEmployeeService _service;

    public EmployeeController(IEmployeeService service)
    {
        _service = service;
    }
}
```

Controller Actions

Index — List Records

```
public IActionResult Index()
{
    return View(_service.GetAll());
}
```

Details — View Record

```
public IActionResult Details(int id)
{
    return View(_service.GetById(id));
}
```

Create — GET

```
public IActionResult Create()
{
    return View();
}
```

Create — POST

```
[HttpPost]
public IActionResult Create(Employee emp)
{
}
```



```
if (ModelState.IsValid)
{
    _service.Add(emp);

    return RedirectToAction("Index");
}

return View(emp);
}
```

Edit — GET

```
public IActionResult Edit(int id)
{
    return View(_service.GetById(id));
}
```

Edit — POST

```
[HttpPost]
public IActionResult Edit(Employee emp)
{
    if (ModelState.IsValid)
    {
        _service.Update(emp);

        return RedirectToAction("Index");
    }

    return View(emp);
}
```



Delete — GET

```
public IActionResult Delete(int id)
{
    return View(_service.GetById(id));
}
```

Delete — POST

```
[HttpPost, ActionName("Delete")]
public IActionResult DeleteConfirmed(int id)
{
}
```

```
_service.Delete(id);  
  
return RedirectToAction("Index");  
}
```

9. Generate Views

Views are strongly-typed Razor pages:

- ✓ Index.cshtml
- ✓ Create.cshtml
- ✓ Edit.cshtml
- ✓ Details.cshtml
- ✓ Delete.cshtml

These may be created using:

- ✓ Visual Studio scaffolding
- or
- ✓ Manual Razor view creation

Views bind to model properties and submit form data back to controller actions.

10. JSON Cycle Handling

If models contain navigation properties (e.g., Employee → Department), circular references may occur.

To prevent:

```
builder.Services.AddControllersWithViews()
```

```
.AddJsonOptions(o =>
```

```
o.JsonSerializerOptions.ReferenceHandler = ReferenceHandler.IgnoreCycles);
```

or apply [JsonIgnore] on specific properties when required.

11. Query Execution Reminder

EF Core queries execute only when materialized:

- ✓ ToList()
- ✓ FirstOrDefault()
- ✓ SingleOrDefault()
- ✓ Count()

This is called **deferred execution**.

12. MVC CRUD Execution Flow

1. Browser sends request
2. Controller receives request
3. Controller calls Service/Repository
4. Service uses DbContext
5. DbContext interacts with database
6. EF Core generates SQL
7. Result returned to controller
8. Controller passes data to View
9. View displays results

This ensures **layered, maintainable architecture**.

13. Migration Command Meaning (Quick Recall)

Command	Purpose
Add-Migration	Generates migration script
Update-Database	Applies migration to DB
-Verbose	Shows executed SQL

14. Common Issues & Checks

If CRUD fails:

- ✓ Migration applied?
 - ✓ Connection string correct?
 - ✓ DbSet declared?
 - ✓ Service registered in Program.cs?
 - ✓ Dependency injection correct?
 - ✓ Validation errors resolved?
 - ✓ JSON cycle handled?
-